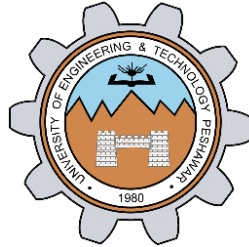


Implementation of 8 bit Ring Counter

LAB # 06



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

14th April 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives:

This lab will enable students to:

- To become familiarized with behavior level modeling
- To be able to implement sequential circuits using Verilog
- To Implement an 8 Bit Ring Counter on Spartan 6 FPGA starter kit.

Software used:

- ModelSim

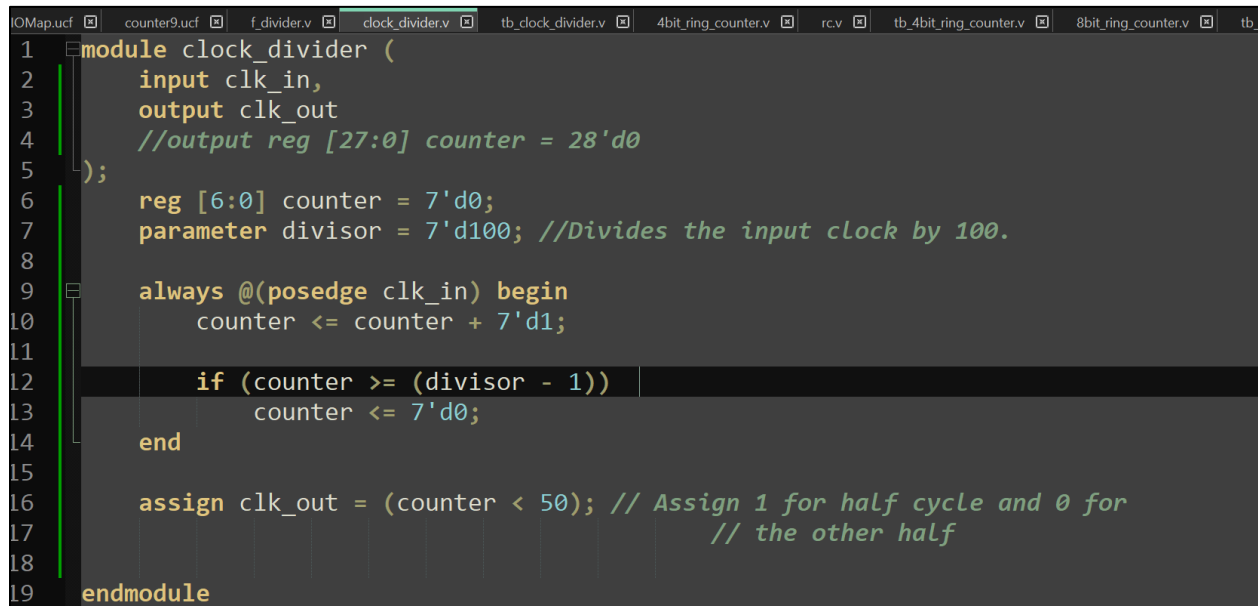
Clock Divider:

A clock divider is a circuit that takes an input clock signal and generates an output clock signal with a lower frequency. These circuits are commonly used in digital systems for various purposes, such as generating slower clock domains, creating timing signals, or driving sequential logic elements.

For simulation purposes, I have generated a 100 GHz signal and divided it by 100 to get 1 GHz output signal.

Code:

DUT Code:



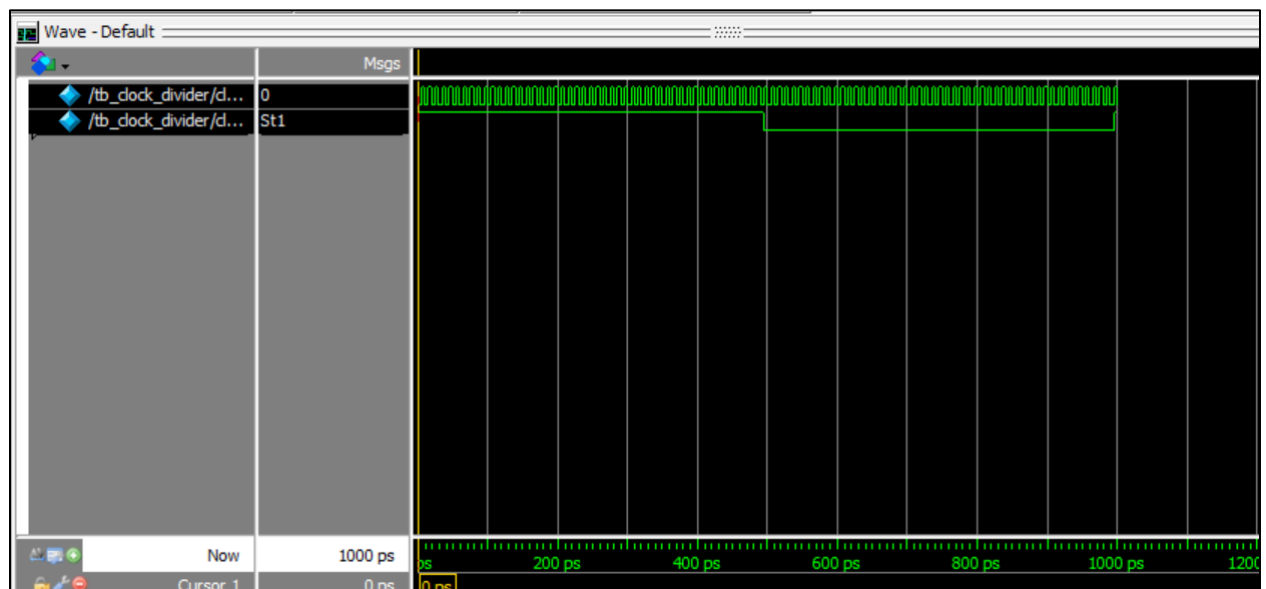
```
1 module clock_divider (  
2     input clk_in,  
3     output clk_out  
4     //output reg [27:0] counter = 28'd0  
5 );  
6     reg [6:0] counter = 7'd0;  
7     parameter divisor = 7'd100; //Divides the input clock by 100.  
8  
9     always @(posedge clk_in) begin  
10         counter <= counter + 7'd1;  
11  
12         if (counter >= (divisor - 1))  
13             counter <= 7'd0;  
14     end  
15  
16     assign clk_out = (counter < 50); // Assign 1 for half cycle and 0 for  
17                                     // the other half  
18  
19 endmodule
```

Test bench Code:

```
IOMap.ucf | counter9.ucf | f_divider.v | clock_divider.v | tb_clock_divider.v | 4bit_ring_counter.v | rc.v | tb_4bit_v

1  module tb_clock_divider;
2      reg clk_in = 0;
3      wire clk_out;
4
5      // Instantiate the clock divider module
6      clock_divider uut (.clk_in(clk_in), .clk_out(clk_out));
7
8      always #5 clk_in = ~clk_in; //100 GHz signal
9
10 endmodule
```

Output:



Task 1:

Implement 4-bit Ring Counter

Code:

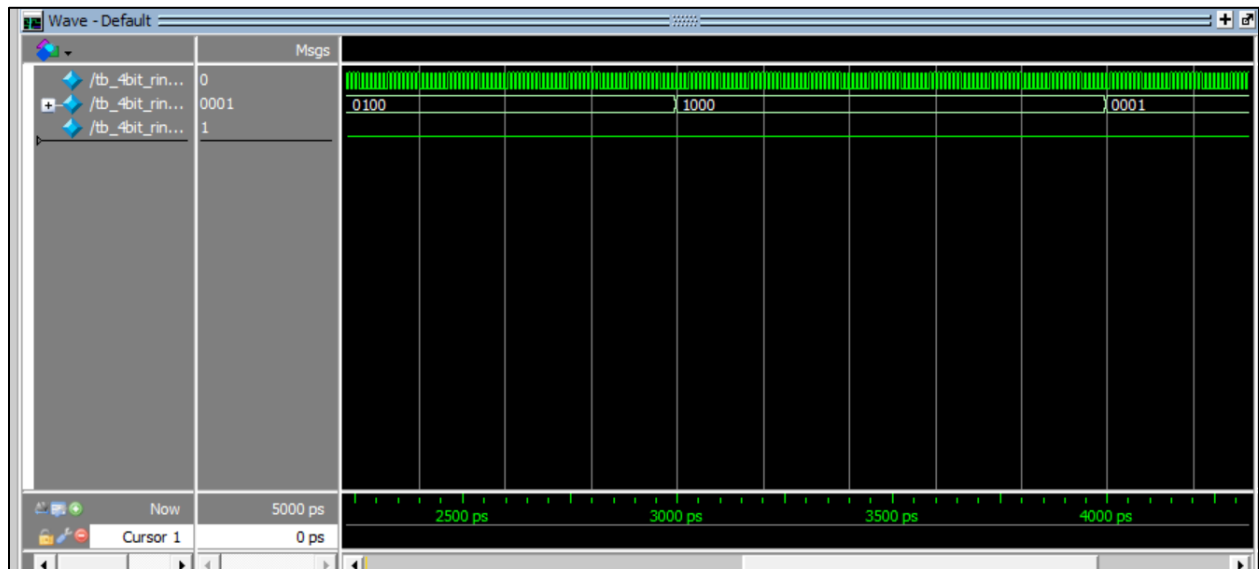
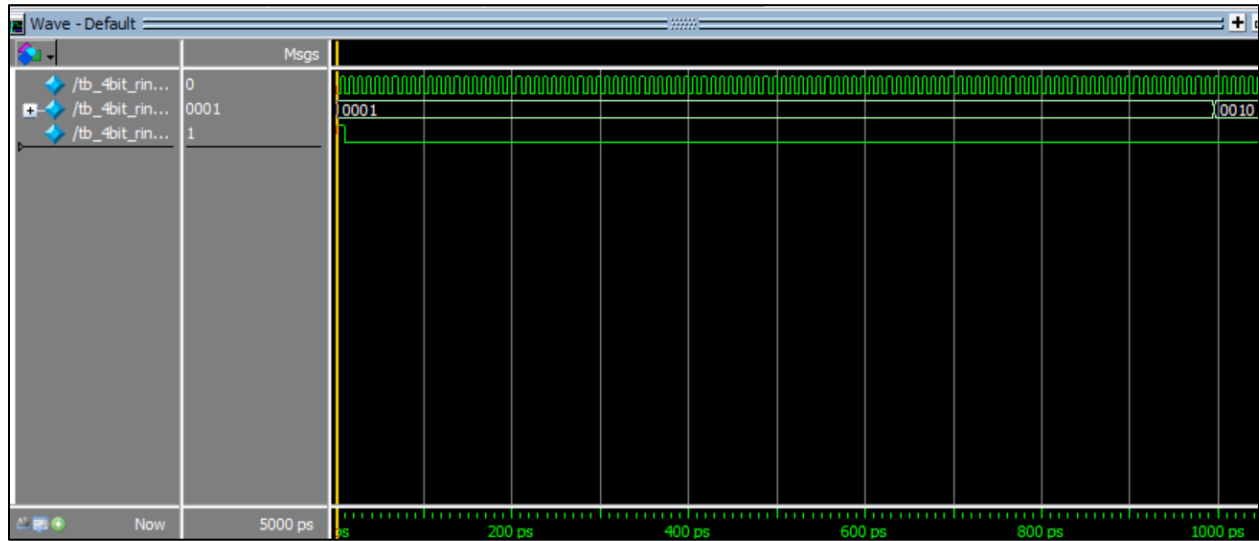
DUT Code:

```
1 module four_bit_ring_counter (
2     input clk,
3     input reset,
4     output reg [3:0] out
5 );
6     wire clk_out;
7
8     clock_divider clk_div(.clk_in(clk), .clk_out(clk_out));
9
10    always @(posedge clk_out) begin
11        if(reset) begin
12            out = 4'b0001;
13        end
14        else begin
15            out = {out[2:0], out[3]};
16        end
17    end
18 endmodule
```

Test bench Code:

```
1 module tb_4bit_ring_counter ();
2     reg clk, reset;
3     wire [3:0] out ;
4
5     four_bit_ring_counter dut(clk, reset, out);
6
7     always #5 clk = ~clk; //100 GHz signal
8
9     initial begin
10        clk = 0;
11        reset = 1;
12
13        #10
14        reset = 0;
15
16    end
17
18 endmodule
```

Output:



Task 2:

Implement 8-bit Ring Counter

Code:

DUT Code:

```
clock_divider.v | tb_clock_divider.v | 4bit_ring_counter.v | tb_4bit_ring_counter.v | 8bit_ring_counter.v | tb_8bit_ring_counter.v |
1  module eight_bit_ring_counter (
2      input clk,
3      input reset,
4      output reg [7:0]out
5  );
6
7      always @(posedge clk) begin
8          if(reset) begin
9              out = 8'b00000001;
10             end
11             else begin
12                 out = {out[6:0], out[7]};
13             end
14         end
15     endmodule
16
```

Test bench Code:

```
clock_divider.v | tb_clock_divider.v | 4bit_ring_counter.v | tb_4bit_ring_counter.v | 8bit_ring_counter.v | tb_8bit_ring_counter.v |
1  module tb_8bit_ring_counter ();
2      reg clk, reset;
3      wire [7:0]out ;
4
5      eight_bit_ring_counter dut(clk, reset, out);
6
7      always #5 clk = ~clk; //100 GHz signal
8      initial begin
9          clk = 0;
10         reset = 1;
11
12         #10
13         reset = 0;
14     end
15 endmodule
```

Output:



Task 3:

For both the counters display the count in the seven-segment display.

Code:

DUT Code: (For 4-bit Ring Counter)

```
1 module task3(  
2     input clk,  
3     input reset,  
4     output reg [6:0]Out  
5 );  
6 wire [3:0]ctr_out;  
7 four_bit_ring_counter counter4(clk, reset, ctr_out);  
8  
9 always @(*)  
10  
11     case(ctr_out)        // gfedcba  
12         4'b0001: Out = 7'b1111001;  
13         4'b0010: Out = 7'b0100100;  
14         4'b0100: Out = 7'b0110000;  
15         4'b1000: Out = 7'b0011001;  
16         default: Out = 7'b1111001;  
17     endcase  
18  
19  
20 endmodule
```

Test bench Code: (For 4-bit Ring Counter)

```
clock_divider.v | tb_clock_divider.v | 4bit_ring_counter.v | tb_4bit_ring_counter.v | 8bit_ring_counter.v | tb_8bit_ring_counter.v | task3.v | tb_task3.v
1  module tb_task3();
2      reg clk, reset;
3      wire [6:0]out;
4
5      task3 t3(clk, reset, out);
6
7      always #5 clk = ~clk; //100 GHz signal
8
9      initial begin
10         clk = 0;
11         reset = 1;
12
13         #10
14         reset = 0;
15     end
16 end
17
18 endmodule
```

Output: (For 4-bit Ring Counter)



Code:

DUT Code: (For 8-bit Ring Counter)

```
1 module task3_b(  
2     input clk,  
3     input reset,  
4     output reg [6:0]Out  
5 );  
6     wire [7:0]ctr_out;  
7     eight_bit_ring_counter counter4(clk, reset, ctr_out);  
8  
9     always @(*)  
10        case(ctr_out)           // gfedcba  
11            8'b00000001: Out = 7'b1111001;  
12            8'b00000010: Out = 7'b0100100;  
13            8'b00000100: Out = 7'b0110000;  
14            8'b00001000: Out = 7'b0011001;  
15            8'b00010000: Out = 7'b0010010;  
16            8'b00100000: Out = 7'b0000010;  
17            8'b01000000: Out = 7'b1111000;  
18            8'b10000000: Out = 7'b0000000;  
19            default: Out = 7'b1111111;  
20        endcase  
21 endmodule
```

Test bench Code: (For 8-bit Ring Counter)

```
1 module tb_task3_b();  
2     reg clk, reset;  
3     wire [6:0]out;  
4  
5     task3_b t3(clk, reset, out);  
6  
7     always #5 clk = ~clk; //100 GHz signal  
8  
9     initial begin  
10        clk = 0;  
11        reset = 1;  
12  
13        #10  
14        reset = 0;  
15    end  
16  
17  
18 endmodule
```

Output: (For 8-bit Ring Counter)

